

Day 10

Managing Users and Permissions in Linux

Passwords, Account Locking, Shadow File and Groups: explained in simple language

What you will learn	Simple meaning
Password security	How to keep accounts safe with strong passwords
passwd and chage	Change, lock, unlock and expire passwords; check account details
/etc/shadow	The secret file that stores passwords and their time rules
Groups	Primary and secondary groups, creating, changing and deleting them
Files	/etc/group and /etc/gshadow explained field by field

Tip: Practice only on a practice machine or virtual machine (VM). Commands with **sudo** change the system.

1. Simple Terms You Must Know

Term	Meaning in simple words
Encrypted password	Password turned into unreadable code, so nobody can read the real password.
Lock an account	Block login for that user, but keep the account and its files.
Password expiry	The date after which the old password stops working and must be changed.
Epoch	The starting date of computer time: 1 January 1970. Linux counts days from this date.
Primary group	The main group of a user. Files the user creates belong to this group.
Secondary group	Extra groups a user belongs to, to get more access.
GID	Group ID: the number of a group.
Root privileges	Administrator power. You get it by using sudo or by being root.
Syntax	The correct way to write a command.
Option / Flag	A small extra with a dash, like -l, that changes how a command works.

Reading syntax: In syntax, words in [] are optional. Words like *username* are placeholders: replace them with a real name.

2. Importance of Password Security

A password is the **first lock** on a user account. A weak password is like a weak lock: anyone can break in and damage the system or steal data.

Good practices

- Use **at least 8 characters**, mixing capital letters, small letters, numbers and special symbols (for example: Tr33@Sky#9).
- **Change passwords from time to time**, so a leaked password stops working.
- **Never share** your password with anyone.
- Do not use easy words like your name, birthday or *password123*.

3. The passwd Command

Use: Changes a user's password and controls password settings (lock, unlock, expire).

Syntax: `passwd [OPTIONS] [USER]`

If you write no user name, the command works on **your own** account. To change another user's password you need root privileges (use sudo).

Command	What it does
<code>passwd</code>	Change your own password.

Command	What it does
<code>sudo passwd username</code>	Change another user's password.
<code>sudo passwd -l username</code>	Lock the account: the user cannot log in with the password.
<code>sudo passwd -u username</code>	Unlock the account: login works again.
<code>sudo passwd -e username</code>	Expire the password: the user must set a new password at the next login.

```
passwd           # change my password
sudo passwd sourabh # change sourabh's password
sudo passwd -l sourabh # lock sourabh
sudo passwd -u sourabh # unlock sourabh
sudo passwd -e sourabh # force password change at next login
```

Real-life use: `-e` is useful when you create a new user with a temporary password. The user is forced to choose a private password at first login.

4. Account Locking and Expiration

Locking stops unauthorised logins (for example, when an employee leaves or an account looks suspicious). **Expiration** makes sure unused or old accounts do not stay active forever.

Lock an account

```
sudo passwd -l username
```

Behind the scenes, Linux puts a `!` in front of the encrypted password in `/etc/shadow`. Because of that mark, the password no longer matches. Unlocking removes the mark.

Command: `chage -l` (check account details)

chage means **change age**: it controls the age of a password. The option `-l` means list: it only shows the details.

Syntax: `sudo chage -l username`

```
sudo chage -l sourabh
```

Example output:

```
Last password change           : Oct 06, 2026
Password expires                : never
Password inactive              : never
Account expires                : never
Minimum number of days between change: 0
Maximum number of days between change: 99999
Number of days of warning before expiry: 7
```

chage option	Meaning	Example
-l	Show (list) the details	chage -l sourabh
-M days	Set the Maximum days a password can be used	sudo chage -M 90 sourabh
-m days	Set the minimum days before it can be changed again	sudo chage -m 7 sourabh
-W days	Days of Warning before expiry	sudo chage -W 7 sourabh
-E date	Account Expiry date (YYYY-MM-DD)	sudo chage -E 2026-12-31 sourabh

Extra: The commands in the 'chage option' table (except -l) are extra, to help you set a password policy. They are a good practice.

5. Understanding /etc/shadow

The file /etc/shadow stores **encrypted passwords** and time rules for each account. Only root can read it, for safety.

```
sudo cat /etc/shadow
```

Example line:

```
sourabh:$6$abc...xyz:20000:0:99999:7:::
```

Each line has fields separated by a colon (:):

Field	Example	Meaning in simple words
username	sourabh	The login name.
password	\$6\$abc...	The encrypted password. A ! in front (or only !) means the account is locked .
last_change	20000	Days since 1 Jan 1970 when the password was last changed.
min_days	0	Minimum days to wait before the password can be changed again.
max_days	99999	Maximum days the password can be used before it must be changed.
warn	7	How many days before expiry the user gets a warning.
inactive	(empty)	Days after the password expires before the account is locked.
expire	(empty)	The date when the whole account stops working (in days since 1 Jan 1970).

Remember: An empty field means 'no limit set'. There is also one more reserved field at the end that Linux does not use yet.

6. Introduction to Linux Groups

A **group** is a team of users. Instead of giving access to 20 people one by one, give access to the group once. All members get it.

Group type	Meaning	Example
Primary group	The user's default main group, set when the user is created. New files are owned by this group.	User sourabh has primary group sourabh
Secondary group	Extra groups the user is added to for more access.	sourabh is also in group developers

To see both kinds of groups for a user, use `id username` (shows the primary group first) or `groups username`.

7. Group Files: `/etc/group` and `/etc/gshadow`

`/etc/group` (public group information)

```
group_name:x:GID:members
```

Example:

```
dev_team:x:1005:alice,bob
```

Field	Meaning in simple words
group_name	The name of the group.
x	Placeholder for a group password. The real one (if any) is in <code>/etc/gshadow</code> .
GID	The group's ID number.
members	Users in the group, separated by commas. (Users whose <i>primary</i> group this is may not appear here.)

`/etc/gshadow` (secure group information)

```
group_name:password:admin_users:members
```

Example:

```
dev_team:::alice:alice,bob
```

Field	Meaning in simple words
group_name	The name of the group.
password	Encrypted group password. Rarely used. A <code>!</code> means no one can use it.
admin_users	Users allowed to manage the group (add or remove members).
members	Normal members of the group.

Compare: `/etc/group` can be read by everyone. `/etc/gshadow` is protected like `/etc/shadow`, so only root can read it.

8. Managing Groups

Task	Command	Syntax
Create a group	groupadd	sudo groupadd group_name
Delete a group	groupdel	sudo groupdel group_name
Rename a group	groupmod -n	sudo groupmod -n new_name old_name
Change group ID	groupmod -g	sudo groupmod -g new_gid group_name

```
sudo groupadd dev_team          # create the group
sudo groupmod -n project_team dev_team  # rename dev_team to project_team
sudo groupmod -g 2000 project_team      # give it GID 2000
sudo groupdel project_team            # delete it
```

Part	What it does
groupmod	Modify (change) an existing group.
-n	New name. Write the new name first, then the old name.
-g	Change the group ID number.

Common mistake: You cannot delete a group if it is the **primary group** of some user. Change that user's primary group first, then delete the group.

9. Managing Group Memberships

Task	Command
Add a user to a group	sudo usermod -aG group_name username
Remove a user from a group	sudo gpasswd -d username group_name
View a user's groups	groups username

```
sudo usermod -aG dev_team alice    # add alice
sudo gpasswd -d alice dev_team     # remove alice
groups alice                       # check
```

Part	What it does
usermod	Modify an existing user.
-a	Append: add the group and keep the user's old groups.
-G	The Group name that follows.
gpasswd	A tool to manage group members and group passwords.

Part	What it does
-d	Delete the user from the group.

Very important: Always write **-aG** together. Using only **-G** removes the user from all their other secondary groups. Also, a user may need to log out and log in again to see the new group.

Viewing group information

```
cat /etc/group          # list of all groups
grep dev_team /etc/group # only one group's line
getent group dev_team    # same, simple way
```

10. Practical Scenario: Create a New Team

Goal: Create a group called `dev_team` and add two users, `alice` and `bob`.

Step 0 (only if users do not exist): create them first.

```
sudo useradd -m -s /bin/bash alice
sudo useradd -m -s /bin/bash bob
```

Step 1: Create the group.

```
sudo groupadd dev_team
```

Step 2: Add users to the group.

```
sudo usermod -aG dev_team alice
sudo usermod -aG dev_team bob
```

Step 3: Verify membership.

```
groups alice

Example output:
alice : alice dev_team

grep dev_team /etc/group
dev_team:x:1005:alice,bob
```

Step 4 (clean up after practice):

```
sudo gpasswd -d alice dev_team
sudo gpasswd -d bob dev_team
sudo groupdel dev_team
sudo userdel -r alice
sudo userdel -r bob
```

11. Quick Revision Cheat Sheet

Command	What it does
<code>passwd</code>	Change own password
<code>sudo passwd username</code>	Change another user's password
<code>sudo passwd -l username</code>	Lock account
<code>sudo passwd -u username</code>	Unlock account
<code>sudo passwd -e username</code>	Force password change at next login
<code>sudo chage -l username</code>	Show password and account age details
<code>sudo cat /etc/shadow</code>	View encrypted passwords and rules
<code>sudo groupadd name</code>	Create group
<code>sudo groupdel name</code>	Delete group
<code>sudo groupmod -n new old</code>	Rename group
<code>sudo groupmod -g gid name</code>	Change group ID
<code>sudo usermod -aG group user</code>	Add user to group
<code>sudo gpasswd -d user group</code>	Remove user from group
<code>groups user</code>	Show user's groups
<code>cat /etc/group</code>	List all groups

Practice questions

- Why is a strong password important? Write 3 rules for a good password.
- What is the difference between **passwd -l** and **passwd -e**?
- What does a **!** in the password field of `/etc/shadow` mean?
- What is the difference between a primary group and a secondary group?
- Which command removes a user from a group?
- Why should we always use **-a** with **-G**?

Summary: Conclusion: Strong passwords, locked or expired accounts, and well-organised groups together keep a Linux system secure and easy to manage.